

四電一丙 數位邏輯實習

第 7 次實習作業報告

組員 1 學號:B812111001 姓名: 張博翔

組員 2 學號: 114310326 黃子軒

組員 1 於此作業中的貢獻:解題、digital、Quartus、協助隊友排錯、作業二、作業報告(二次編排、修改與美編)

組員 2 於此作業中的貢獻:解題、digital、quartus、作業一、作業報告(初步編排、複審內容)

I. 實驗目的

A. 作業 1

設計四位元雙向移位暫存器，其狀態表如下，本題不限定設計方式(VHDL 或拉電路)。

Inputs			Next State			
LSh (Left)	RSh (Right)	Ld (Load)	Q_3^+	Q_2^+	Q_1^+	Q_0^+
0	0	0	Q_3	Q_2	Q_1	Q_0
0	0	1	D_3	D_2	D_1	D_0
0	1	X	SI	Q_3	Q_2	Q_1
1	0	X	Q_2	Q_1	Q_0	SI
1	1	X	Q_3	Q_2	Q_1	Q_0

B. 作業 2

設計 SRAM 並連接先前設計的加法器與比較器，做成累加器，並偵測溢位。

第一步：以 VHDL 設計方式，實作 SRAM 電路，使其可以儲存四組，每組 4 位元數字。

第二步：連接 SRAM 與先前設計的加法器，設計成累加器。(不限定設計方式)

第三步：連接先前設計的比較器，比較每次累加後的結果比前一次更大，若溢位發生則鎖住 SRAM，避免溢位資料存入。(加分)

II. 設計程序

A. 作業 1

1. 第一題不限定設計方法，可以用 VHDL 撰寫或是直接繪製電路圖實作，但是繪製電路費時且因為複雜排除錯誤也花時間，因此使用 VHDL 撰寫。首先依照題目給的狀態表去寫 VHDL，會用到 CLK 語法處理時序邏輯以及利用 if...elsif 判斷控制訊號，設計完成進入模擬驗證。

表 1. 四位元雙向移位暫存器狀態表

LSh (左移)	RSh (右移)	Ld (載入)	動作說明	次態輸出 (Q_3, Q_2, Q_1, Q_0)
0	0	0	保持不變	Q_3, Q_2, Q_1, Q_0
0	0	1	平行載入	D_3, D_2, D_1, D_0
0	1	X	右移	SI, Q_3, Q_2, Q_1
1	0	X	左移	Q_2, Q_1, Q_0, SI
1	1	X	保持不變	Q_3, Q_2, Q_1, Q_0

2. 四位元雙向移位暫存器 VHDL 程式碼設計與講解。

```

1. LIBRARY ieee;
2. USE ieee.std_logic_1164.all;
3.
4. entity dff4 is
5. port (
6.     clk : in std_logic;
7.     LSh : in std_logic;
8.     RSh : in std_logic;
9.     Ld  : in std_logic;
10.    SI  : in std_logic;
11.    D   : in std_logic_vector(3 downto 0);
12.    Q_int : out std_logic_vector(3 downto 0)
13. );
14. end dff4;
15.
16. architecture Behavioral of dff4 is
17.     signal Q_reg : std_logic_vector(3 downto 0) := "0000";
18. begin
19.     process(clk)
20.     begin
21.         if rising_edge(clk) then
22.             if LSh = '0' and RSh = '0' and Ld = '0' then
23.                 Q_reg <= Q_reg;
24.             elsif LSh = '0' and RSh = '0' and Ld = '1' then
25.                 Q_reg <= D;
26.             elsif LSh = '0' and RSh = '1' then
27.                 Q_reg <= SI & Q_reg(3 downto 1);
28.
29.             elsif LSh = '1' and RSh = '0' then
30.                 Q_reg <= Q_reg(2 downto 0) & SI;
31.
32.             elsif LSh = '1' and RSh = '1' then
33.                 Q_reg <= Q_reg;
34.             end if;
35.         end if;
36.     end process;
37.
38.     Q_int <= Q_reg;

```

圖 1. 四位元雙向移位暫存器 VHDL Process

VHDL 程式碼講解：

- 1~2 行是 Library 宣告區，載入資料庫，定義邏輯閘功能與語法。
- 4~14 行是介面定義，定義有輸入訊號 Lsh、Rsh、Ld、Si、D(四位元 3-0)與 CLK，輸出訊號 Q(四位元 3-0)。
- 16~38 行是程式的核心運算區，定義輸入與輸出間關係，如圖先進行偵測時脈的上升邊緣觸，像是如果 Rsh=0,Lsh=0 和 Ld=0，則輸出 Q 保持原態， elsif Rsh=0,Lsh=0 和 Ld=1，則輸出 Q 輸出 D， elsif Rsh=0,Lsh=1，則輸出 Q= SI 和 Q 並在一起等等，模擬實際電路中邏輯閘關係。

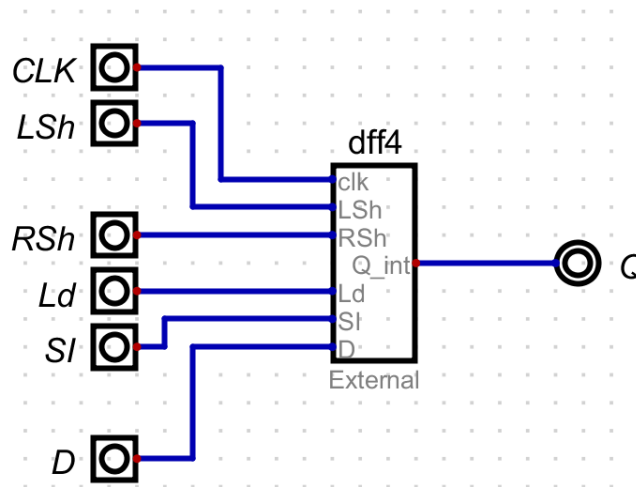


圖 2. 四位元雙向移位暫存器模擬電路

B. 作業 2

1. SRAM 以 VHDL 方式實做，採用高階資料類別進行設計，以 2 維矩陣描述記憶體架構，程式碼相較描述暫存器接線精簡更多。加法器採用的是 lab03 的加減法器、比較器採用的則是 lab04 的比較電路，將其整合在一起實現可偵測溢位的 SRAM 累加電路，若溢位發生則鎖住 SRAM。

表 2. SRAM 電路狀態表

CS	WE	OE	動作
1	1	X	寫入
1	0	1	讀取
其他	任意	任意	無效 / 高阻抗

2. SRAM 電路 VHDL 程式碼設計與講解。

```

-- =====
-- 簡易 4x4 SRAM 模型
-- 4 個記憶體位址，每個位址儲存 4-bit 資料
-- =====

LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

entity SRAM is
port (
    D      : in  std_logic_vector(3 downto 0); -- 資料輸入 (4-bit)
    Address : in  std_logic_vector(1 downto 0); -- 位址輸入 (2-bit，可定址 0~3)
    ChipSelect : in std_logic; -- 晶片選擇：'1' 時才允許讀寫
    WE      : in std_logic; -- 寫入致能 (Write Enable)：'1' 為寫入模式
    OE      : in std_logic; -- 輸出致能 (Output Enable)：'1' 為讀取模式
    DO      : out std_logic_vector(3 downto 0) -- 資料輸出 (4-bit)
);
end SRAM;

architecture SRAM_simple of SRAM is

    -- 定義記憶體型別：4 個元素，每個元素為 4-bit std_logic_vector
    type memory is array (0 to 3) of std_logic_vector(3 downto 0);

    -- 宣告內部記憶體陣列，初始值全部清為 "0000"
    signal SRAM_internal : memory := (others => "0000");

begin

    -- =====
    -- 寫入程序 (Write Process)
    -- 當 ChipSelect = '1' 且 WE = '1' 時，
    -- 將資料 D 寫入 Address 對應的記憶體位置
    -- =====
    process(ChipSelect, WE, Address, D)
    begin
        if (ChipSelect = '1' and WE = '1') then
            -- Address 轉型為 unsigned 再轉為整數，作為陣列索引
            SRAM_internal(to_integer(unsigned(Address))) <= D;
        end if;
    end process;

    -- =====
    -- 讀取邏輯 (Read Logic) — 條件式並行信號指定
    -- 當 ChipSelect = '1'、OE = '1' 且 WE = '0' 時（讀取模式），
    -- 輸出對應位址的記憶體內容；
    -- 否則輸出高阻抗 "ZZZZ"，代表資料匯流排未被驅動
    -- =====
    DO <= SRAM_internal(to_integer(unsigned(Address)))
        when (ChipSelect = '1' and OE = '1' and WE = '0')
        else "ZZZZ"; -- 高阻抗，放開匯流排控制權

end SRAM_simple;

```

圖 3. SRAM 電路 VHDL Process

3. Embedded Circuit，將加減法器、比較器帶入，以下為子電路展開。

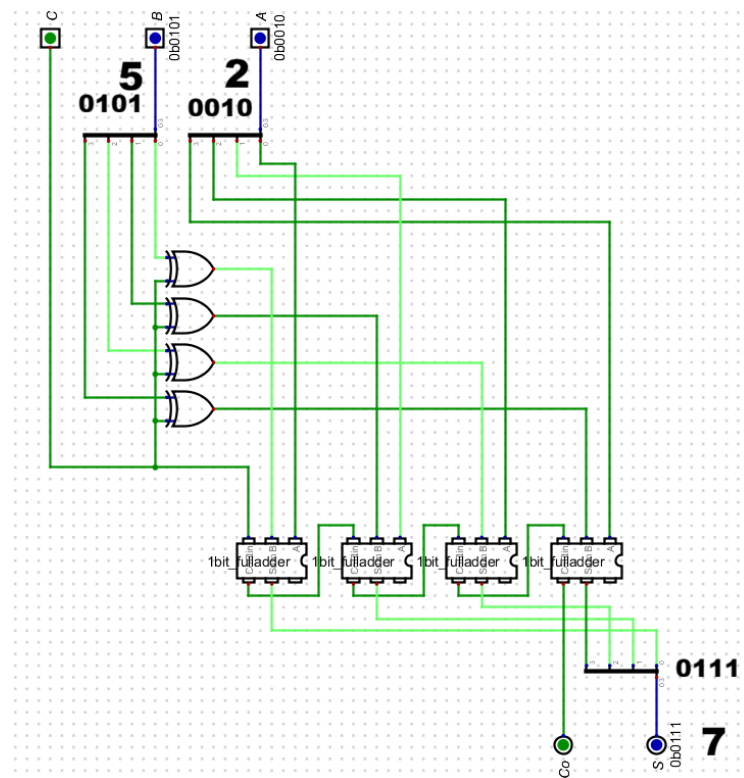


圖 4. 四位元加減法器邏輯電路

將模組串聯合成 4-bit Ripple Carry Adder，為了實現加減法器(Adder-Subtractor)，需讓 C_{in} 與其中一組數值進行 XOR 運算，當 $C_{in}=0$ ，兩數值直接相加；當 $C_{in}=1$ ，則與 C_{in} XOR 運算的數值會取 1 補數，加上 C_{in} 取 2 補數，達成減法效果，設計為了簡化將輸入與輸出位元整併一起精簡畫面，並且後續設計須將 C_{in} 持續接地，只使用加法功能。

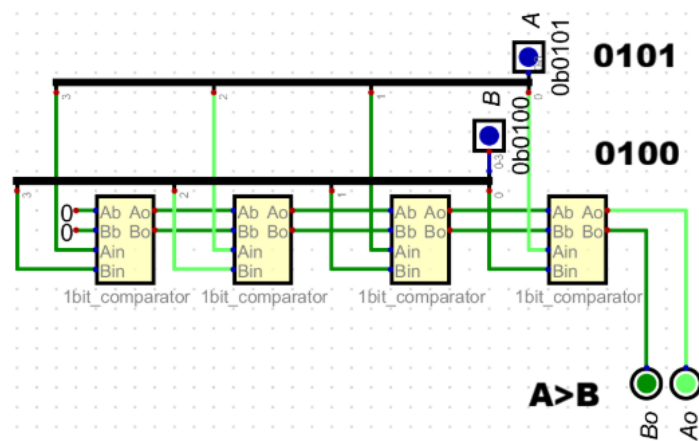


圖 5. 四位元比較器邏輯電路

採用由高位元至低位元逐位比較的方式進行。為避免傳統比較器無法直接表示「等於」的問題使用四輸入兩輸出之一位元比較器作為基本模組，其中 A_o 表示 $A > B$ ， B_o 表示 $B > A$ ，當兩者皆為 0 時代表相等。設計概念為「結果優先傳遞」，每一位元比較器除了接收當前位元輸入 A_{in} 、 B_{in} 外，亦接收來自高位元的比較結

果 A_b 、 B_b 。當 A_b 與 B_b 不同時，代表高位元已決定整體大小關係，此時低位元不需再進行比較，直接將結果傳遞至輸出，僅在 $A_b = B_b = 0$ 的情況下，才由當前位元進行比較判斷。

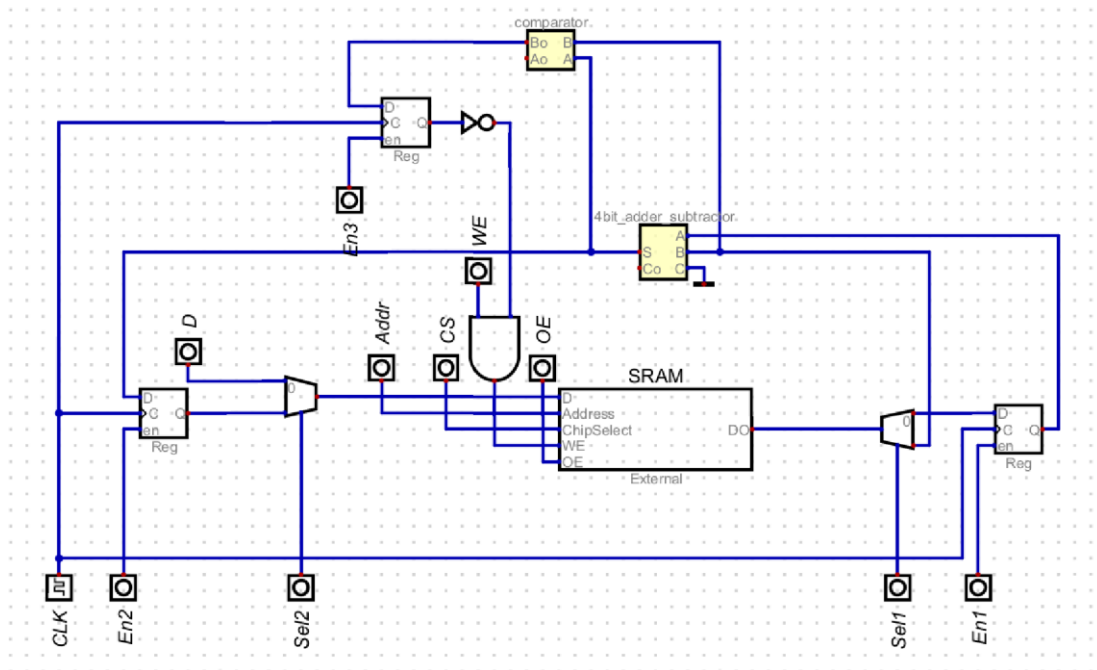


圖 6. SRAM 完整電路(運用 WE 避免 Overflow)

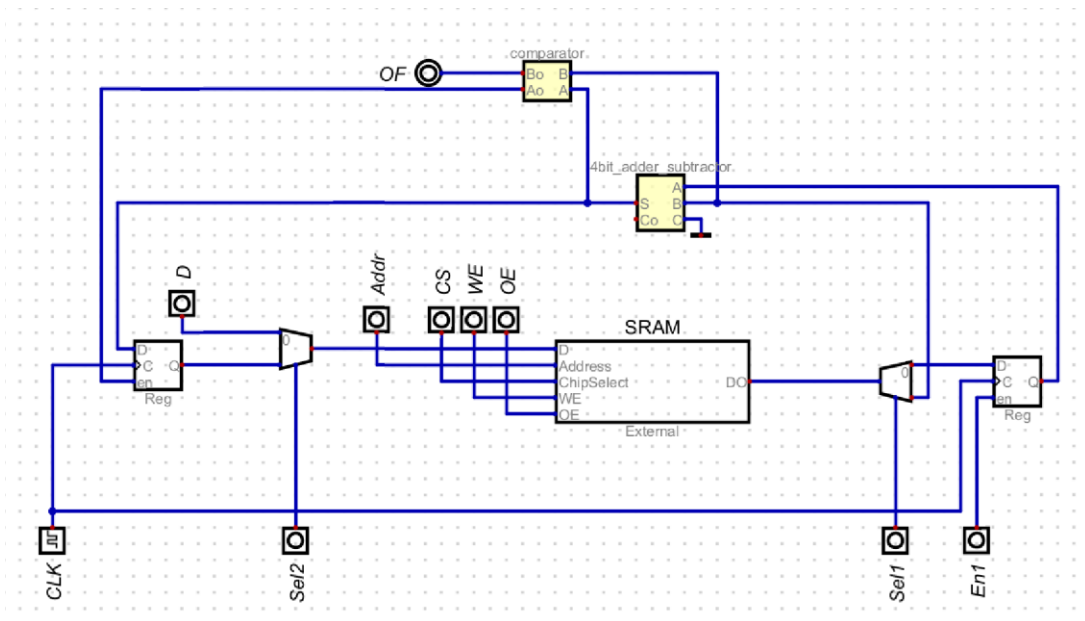


圖 7. SRAM 完整電路(運用 En2 避免 Overflow)

上圖 6、7 架構功能相同，架構為 enable1、2 分別控制 register1、2 存入，register1 負責每次累加的數值，register2 則是累加數值的計算結果，分別透過 sel1 將 SRAM 輸出送到加法器以及 sel2 寫回 SRAM。其中 D 為輸入，由於為記憶體大小為 4×4 ，因此 D 為 4 bits，而 Addr 則是 2 bits。其中 CS 為 Chipselect，控制記

憶體啟動，而 WE、OE 則是分別代表寫入與寫出，兩者不可同時致能，否則解果相衝突，呈現高阻抗狀態。另外，加上比較器來避免 Overflow 錯誤資料讀入，當原輸入加完後卻小於原輸出時，則表示溢位發生，此時會鎖住 SRAM 內數值。

圖 6.使用的是判斷溢位的位元，概念為拿其與寫入做 AND，當溢位發生時，會鎖住 WE 的輸入，避免寫入 SRAM，需多加 1 組 register 存取溢位資料。

圖 7.使用的是正常輸出時會亮的腳位，以此特性來致能存取累加數值的 register2，當 Overflow 發生時因為 register2 關閉，因此會將數值鎖在前一次結果，因此就算可以寫入也是輸入溢位前數值。

其中要注意加法器進位位元應浮接而非直接接地，否則可能會因為溢位發生時 Co 變為 1，直接造成短路。

III. 模擬驗證

A. 作業 1

模擬測試，測試並確認其功能包含平行載入、右移、左移以及保持不變四種主要功能。

1. 當 D 為 0101、Ld 為 1，CLK 從 0 變 1，其餘為 0，輸出 Q 為 D=0101。

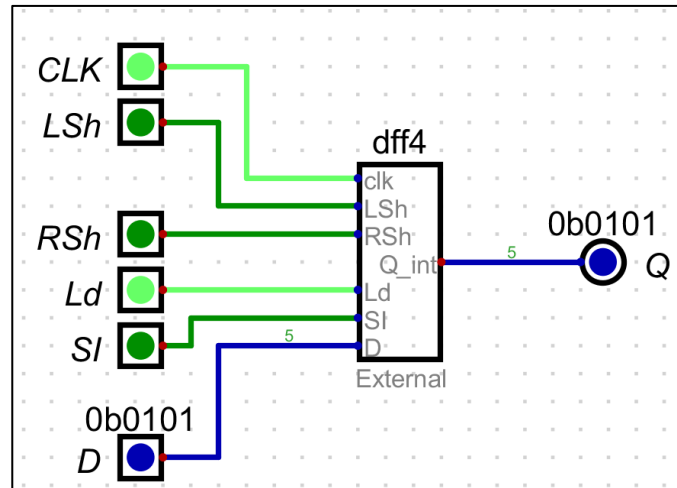


圖 8: 四位元雙向移位暫存器模擬電路(Load)

2. 當 D 為 0101、Rsh 為 1，CLK 從 0 變 1，其餘為 0，輸出 Q 為往右一格(SI 為 0 補 0)=0010。

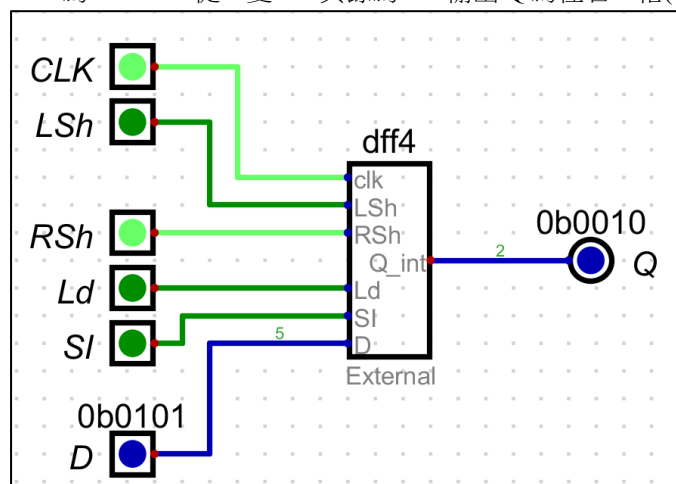


圖 9: 四位元雙向移位暫存器模擬電路(Right Shift, SI = 0)

3. 當 D 為 0101、Lsh 為 1 和 SI 為 1，CLK 從 0 變 1，其餘為 0，輸出 Q 為往左一格(SI 為 1 補 1)=0101。

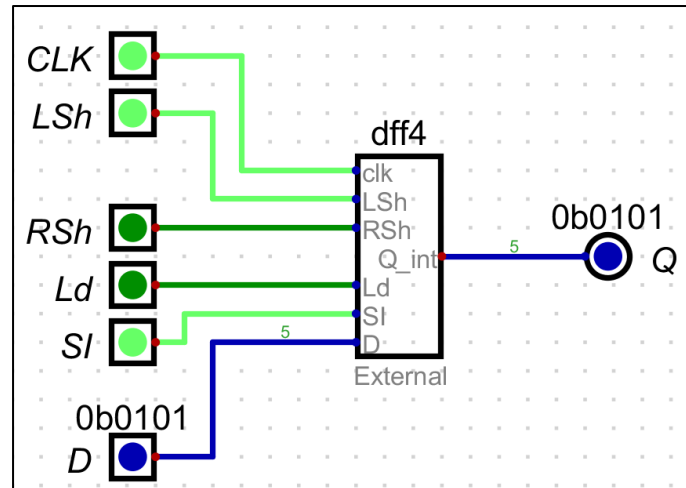


圖 10: 四位元雙向移位暫存器模擬電路(Left Shift, SI = 1)

4. 當 D 為 0111、CLK 從 0 變 1，其餘為 0，輸出 Q 為保持不動=0101。

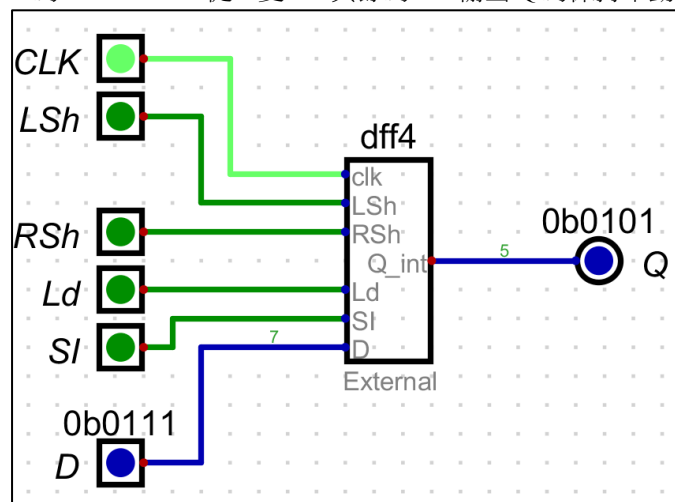


圖 11: 四位元雙向移位暫存器模擬電路(無動作)

B. 作業 2

SRAM 電路測試：10 進位表示，Calculate：起始值(5)+累加值(5)，Result：極限應只能加到 15 (F)

CLK：正緣觸發

SEL：選擇路徑

En：元件致能

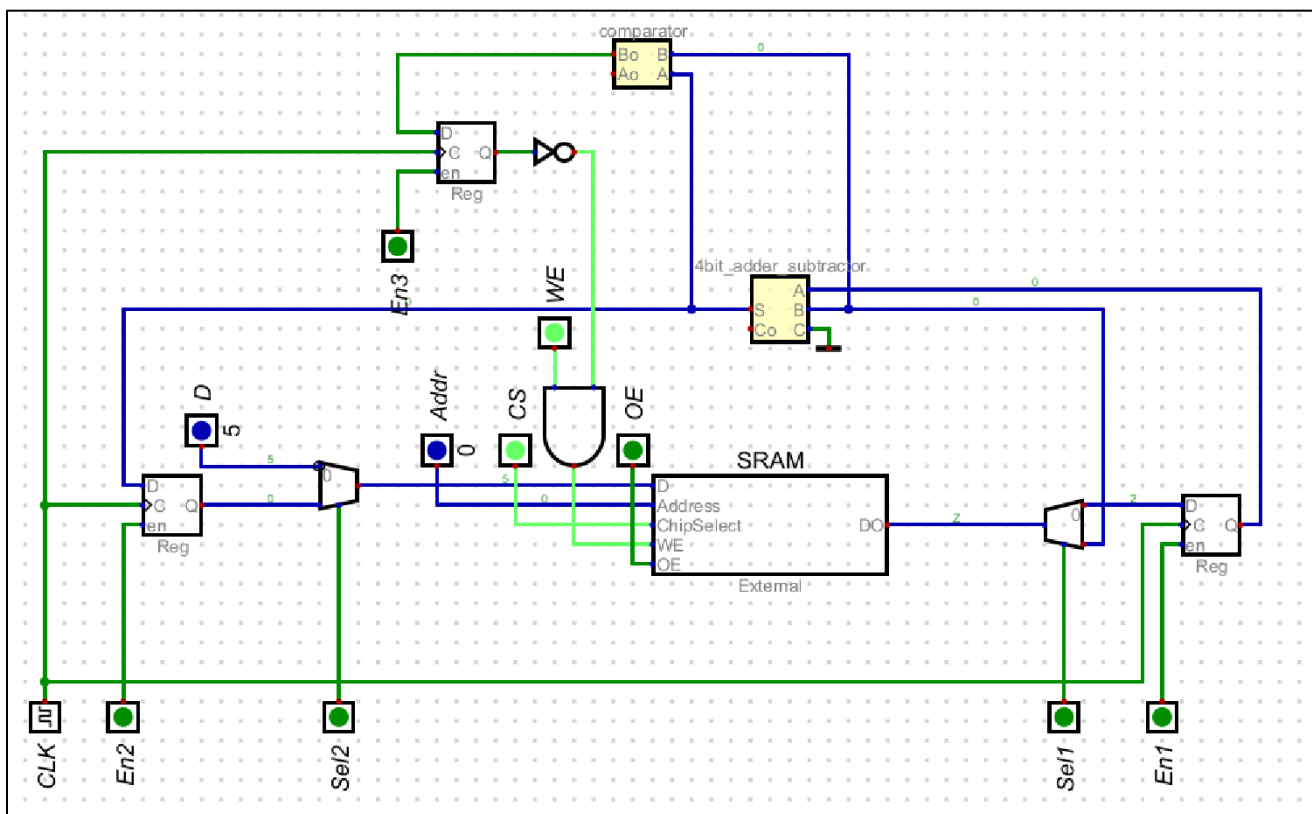


圖 12. Addr0 存入累加值(5) Step1

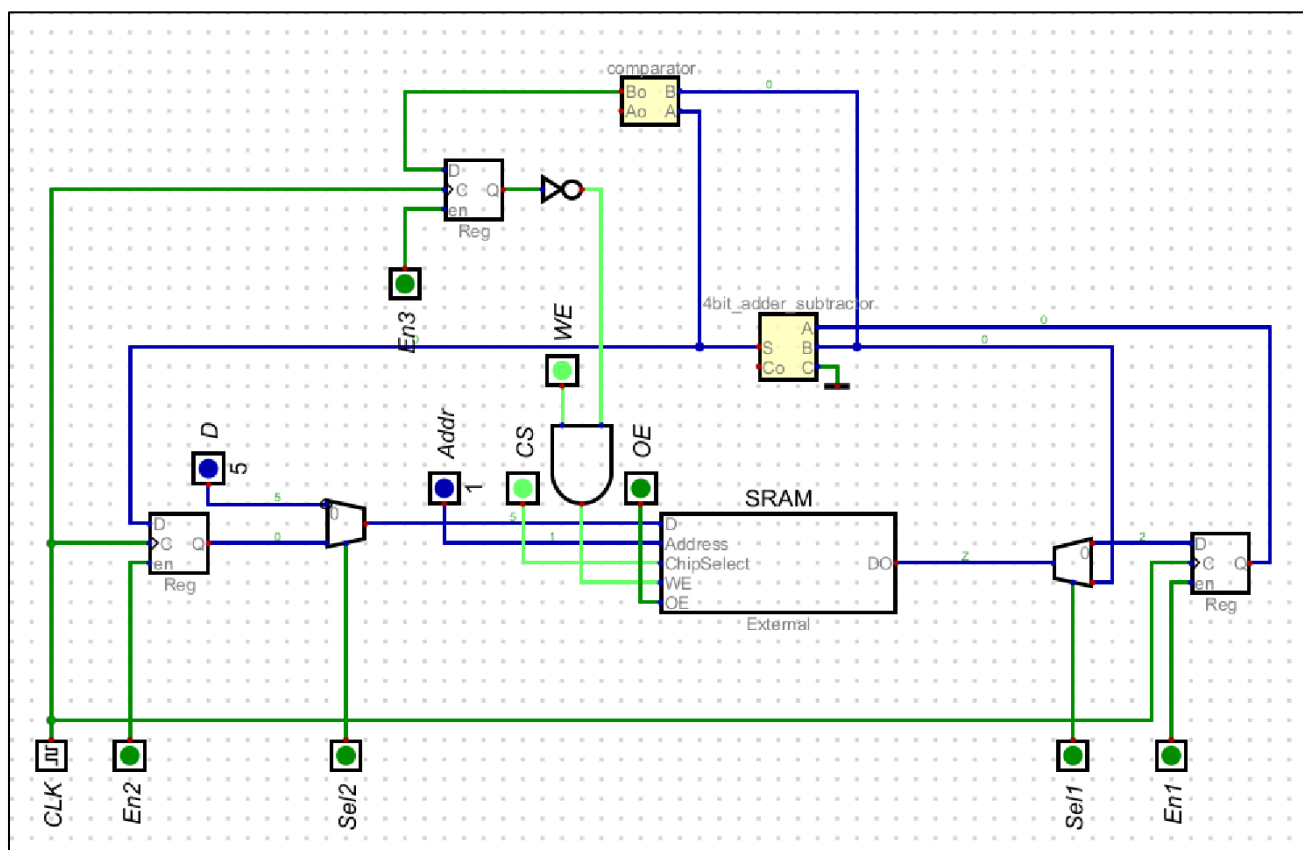


圖 13. Addr1 存入初值(5) Step2

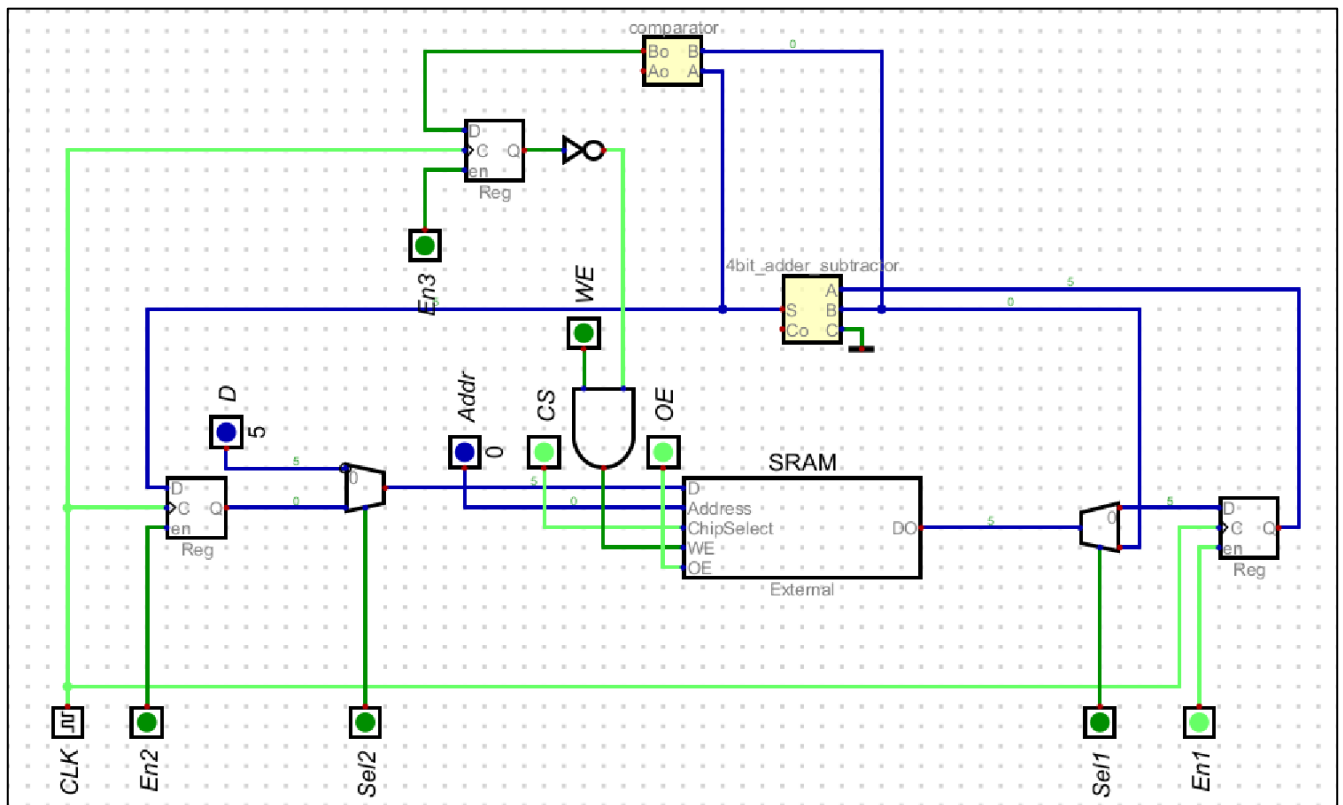


圖 14. 累加值(5)寫入 Register1(5) Step3

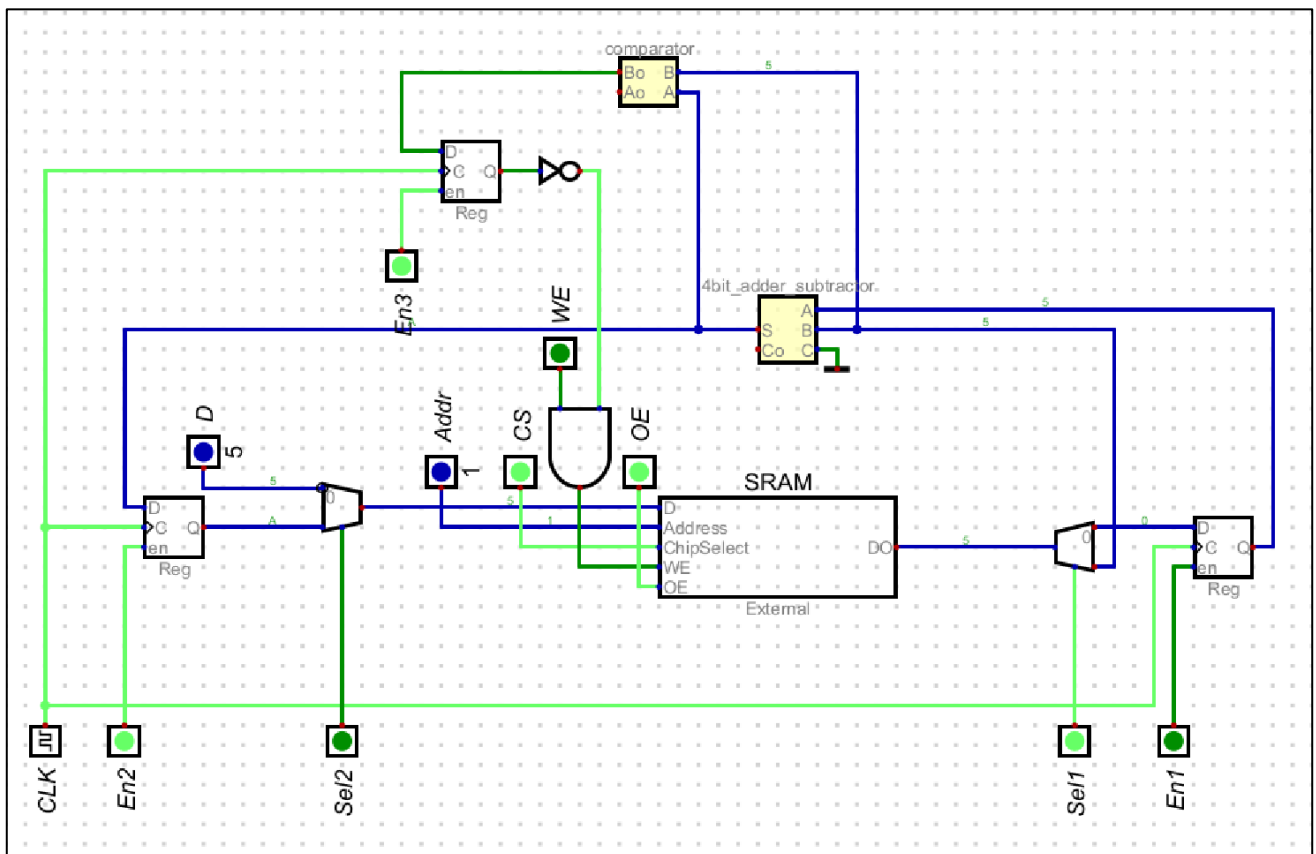


圖 15. 初值(5)經過加法器(+5)後寫入 Register2(A) Step4

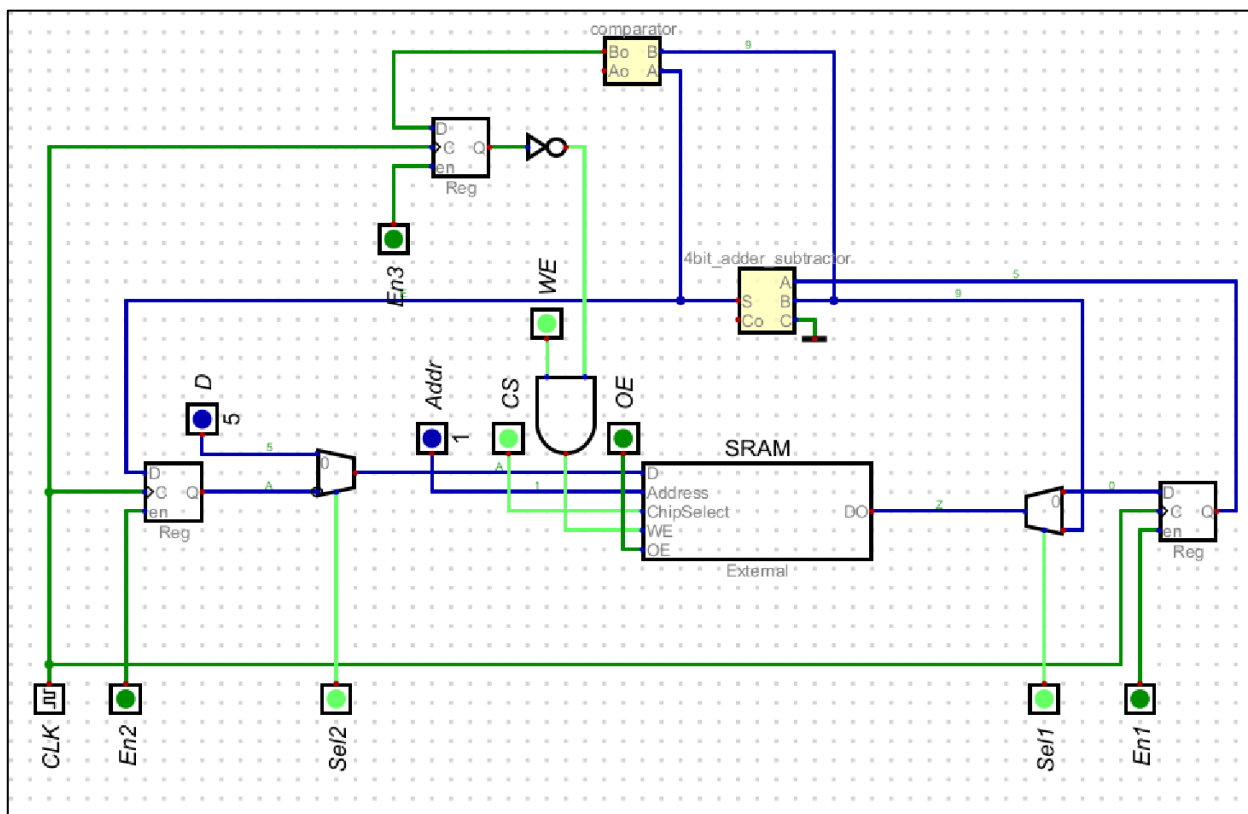


圖 16. 計算結果 Register2(A)寫回 SRAM Step5

重複 Step4 ~ Step5

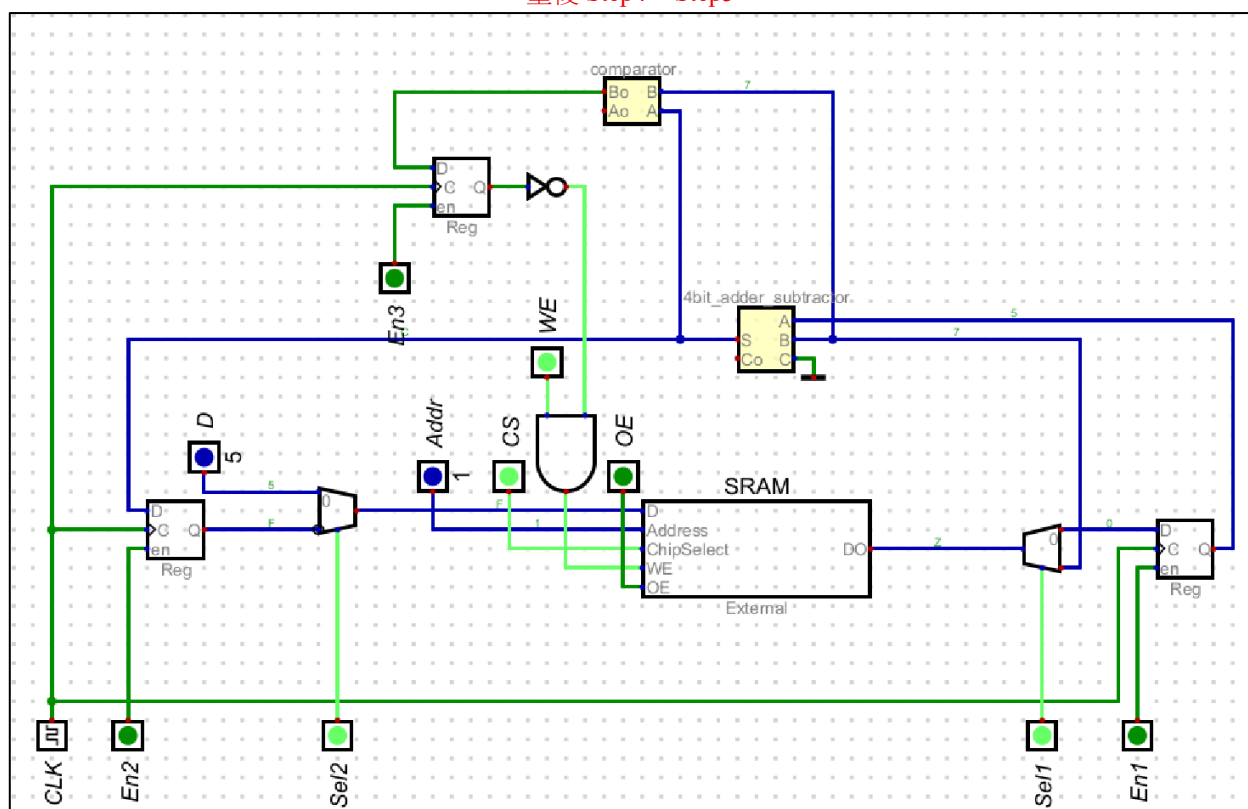


圖 17. 計算結果 Register2(A+5=F)寫回 SRAM Step5

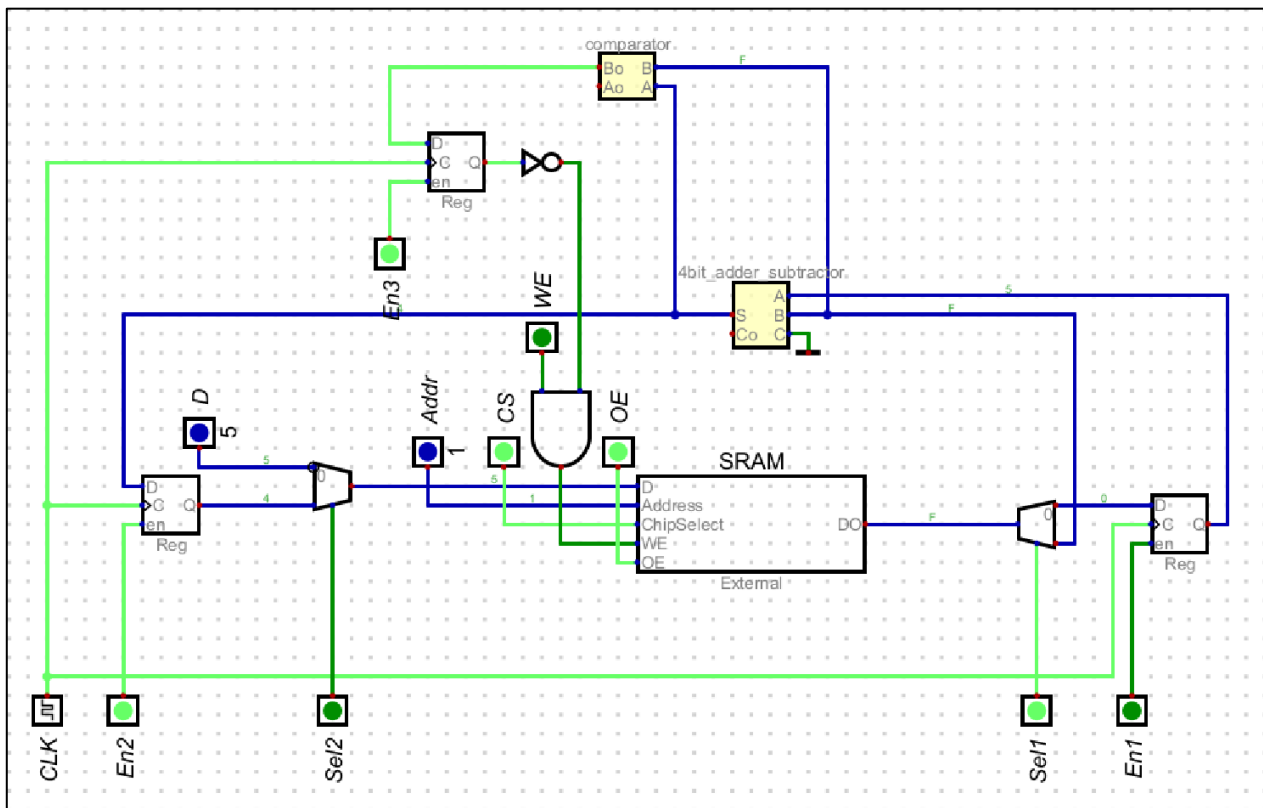


圖 18. 發生溢位，溢位結果($F+5=4$)存入 Register2 Step4

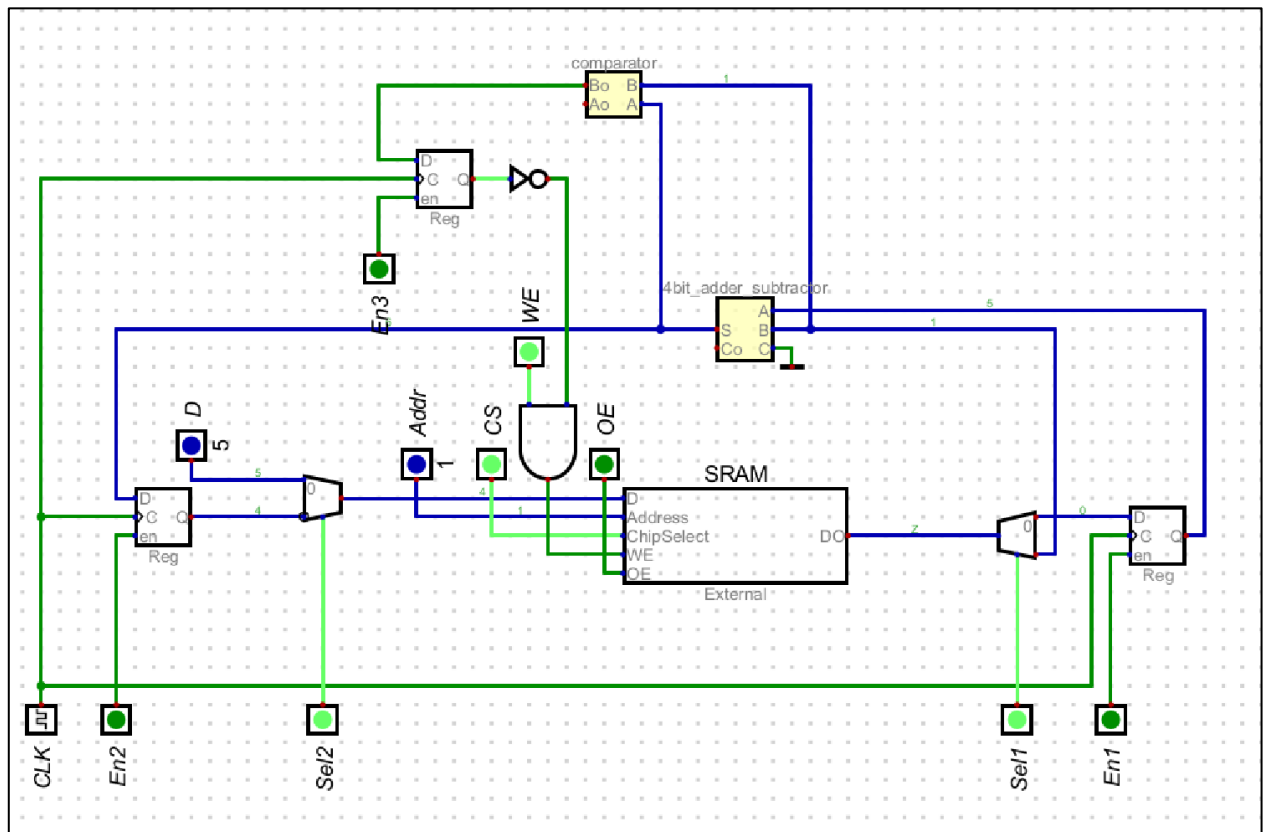


圖 19. WE 已鎖住，結果 Register2(4)無法寫回 SRAM Step5

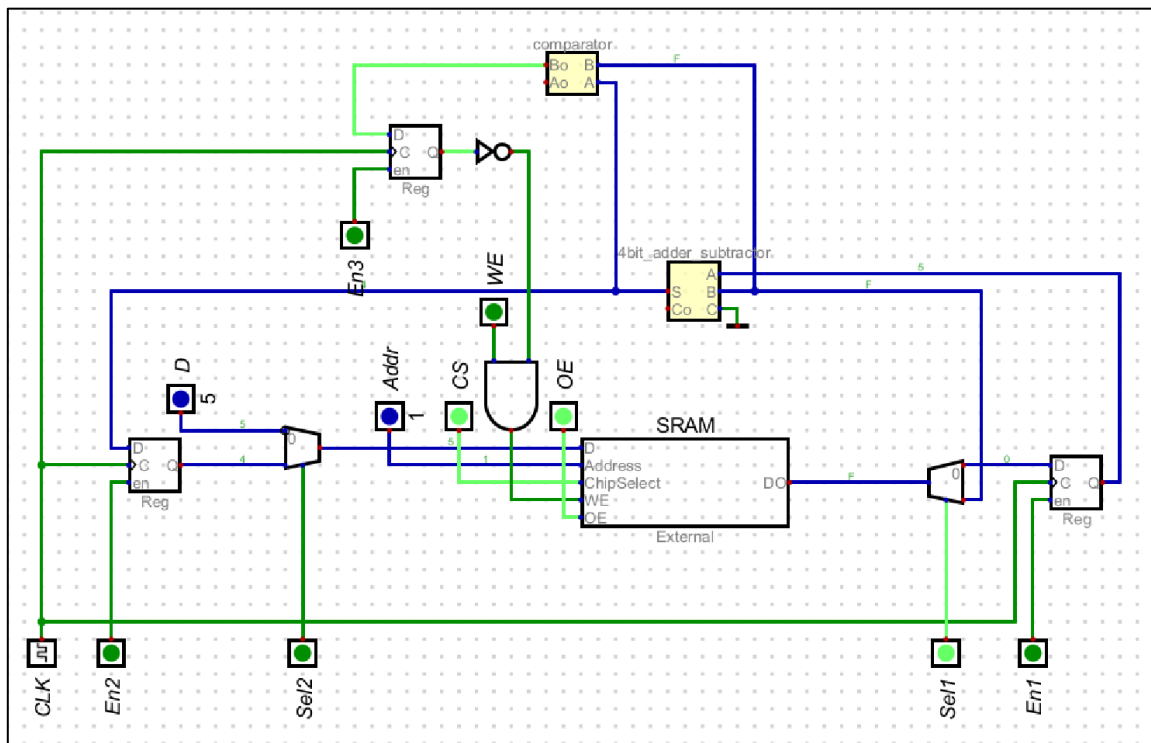


圖 20. 鎖住 SRAM 資料，Addr1 = F(溢位前)

另外以下拿另一個位元做，透過關閉 Register2 達到同樣鎖住 SRAM 效果

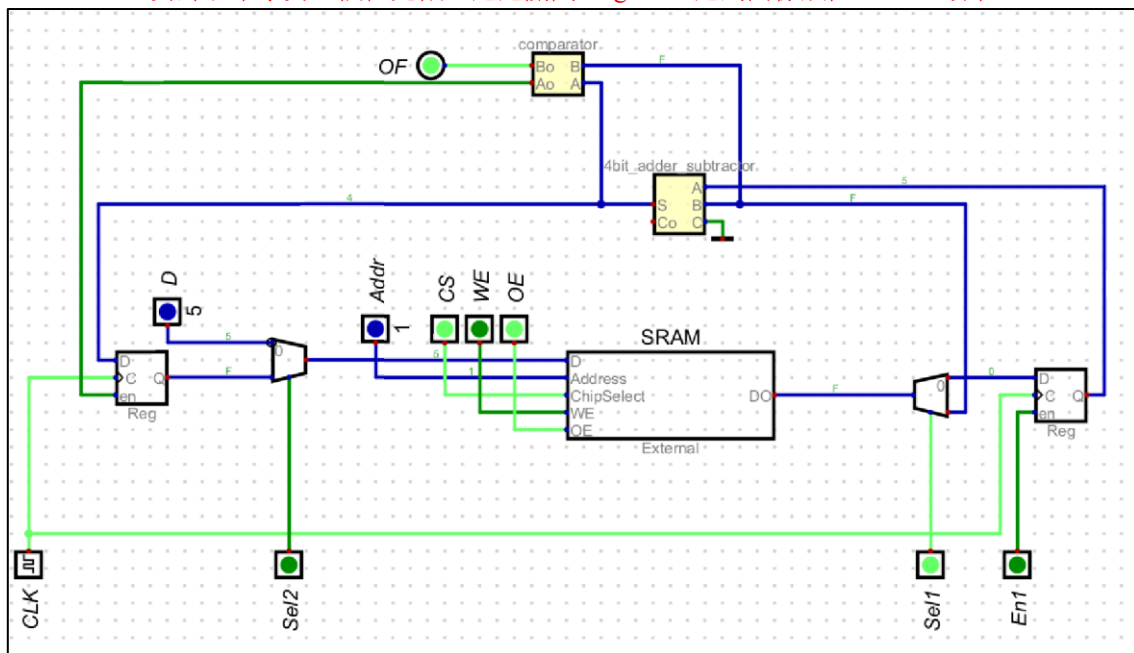


圖 21. 可以發現 Overflow 發生鎖住，Register2 disable，只能重複寫入 F(溢位前)

IV. 實驗結果與成果討論

A. 作業 1

四位元雙向移位暫存器：Input Lsh、Rsh、Ld、Si、D(3~1)、CLK；Output Q(3~1)；Input Input Lsh、Rsh、Ld、Si、D(3~1) 從左到右分別為 SW7、SW6、SW5、SW4、SW3、SW2、SW1、SW0，CLK 放在旁邊的按鈕因為放在開關會震盪，Output Q(3~1)為 LED3、LED2、LED1、LED0，以下為實驗結果。



圖 22. Input:D=1010、Ld=1 CLK=1 / Output:Q=1010



圖 23. Input:D=1010、Rsh=1、SI=1 CLK=1 / Output:Q=1101

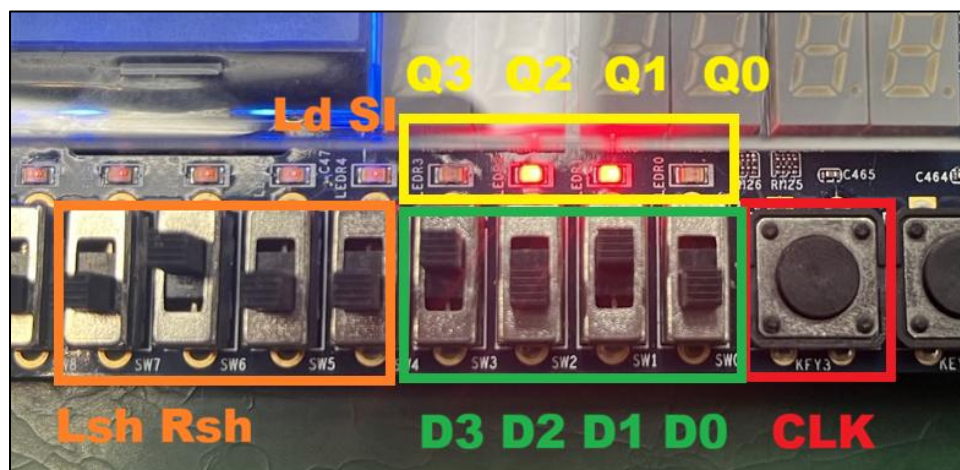


圖 24. Input:D=1010、Rsh=1 CLK=1 / Output:Q=0110

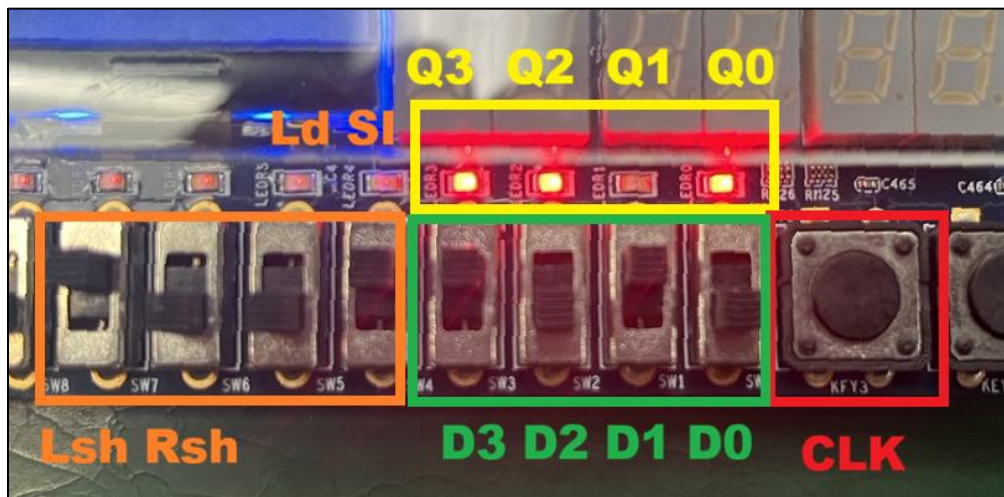


圖 25. Input:D=1010、Lsh=1、SI=1 CLK=1 / Output:Q=1101

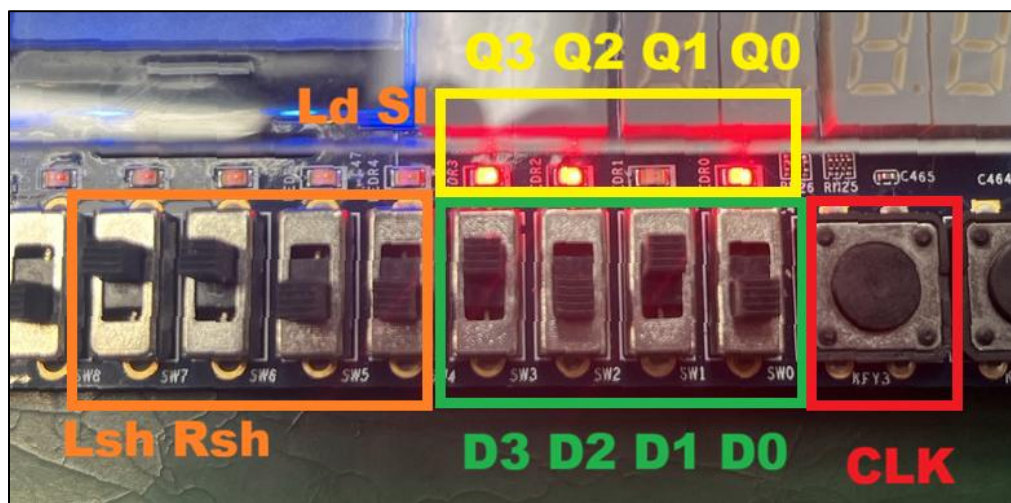


圖 26. Input:D=1010、Lsh=1、Rsh=1 CLK=1 / Output:Q=1101

B. 作業 2

作業二無須燒入電路板實作，只需要模擬驗證。

以上實作兩種方式來控制當輸入超過記憶體容量(溢位)時的解決方式，就電路複雜度而言雖然比較偏好 disable register 的方式，但因為還有其他控制電路，因此最保險且最直觀的方式還是以控制寫入為主。

V. 實驗心得

組員 1：開始與生活中的計算機元件連結，感覺非常有趣且更加容易做連結，希望可以有更多相關主題。

組員 2：這次作業一開始覺得有點難，尤其是時序的部分不太好理解。不過在實際畫波形後，才比較清楚。

參考文獻

- [1] DLE 08 數位邏輯實習 VHDL(I)